ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 7

Базовые методы аутентификации

Рассмотрим процесс аутентификации, хеширование паролей (bcrypt + соль). Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Хеширование паролей

Хеширование паролей — это процесс преобразования пароля в специальный код (хеш) с использованием математической функции. Это позволяет хранить пароль в безопасном зашифрованном виде, чтобы при утечке данных злоумышленники не смогли получить доступ к паролям.

Во время авторизации пароль сначала хешируется и только потом записывается в базу данных. При следующей попытке входа пароль снова переводится в хеш и сравнивается с хешем на сервере. Если хеши совпадают, пользователь получает доступ к системе.

Одним из ключевых преимуществ использования хеш-функций — односторонность — возможность только зашифровать исходные данные, но не расшифровать хеш обратно.

Среди популярных алгоритмов шифрования, применимых для хеширования паролей, можно выделить следующие:

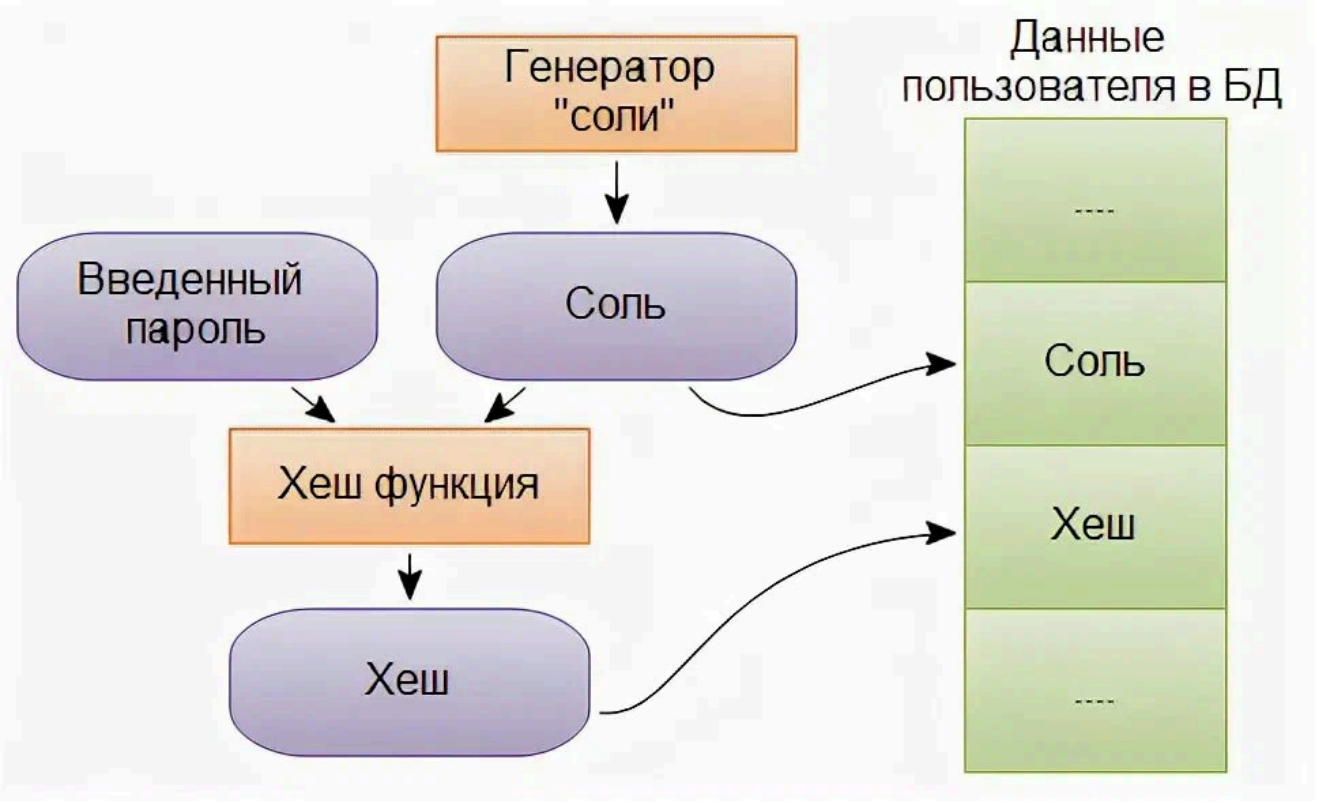
- **SHA-256 и SHA-512** — длина хеша может составлять 256 или 512 бит. Более устойчивы к коллизиям, чем SHA-1 и MD5, но при этом всё ещё относительно быстрые на современных процессорах и GPU.
- **bcrypt** — алгоритм на основе шифра Blowfish, создан специально для хеширования паролей. Имеет настраиваемый параметр «cost», который задаёт число раундов хеширования.

- **Argon2** — современный алгоритм, победивший в конкурсе по выбору нового стандарта по хешированию паролей (Password Hashing Competition). Обладает гибкими настройками: позволяет регулировать время исполнения, объём используемой памяти и параллелизм вычислений, что делает его одним из самых сложных для перебора.

Соль

Соль — случайная строка, которая добавляется к паролю перед его хешированием. Используется для усложнения определения прообраза хэш-функции методом перебора по словарю возможных входных значений, включая атаки с использованием радужных таблиц.

Общий вид применения соли при сохранении пароля пользователя в системе можно изобразить следующим образом:



Bcrypt

Одно из ключевых преимуществ алгоритма bcrypt заключается в том, что он использует соль "под капотом", что позволяет использовать его как единственный механизм шифрования, обеспечивая высокий уровень как безопасности, так и скорости.

Для установки bcrypt в Nodejs, необходимо выполнить команду:

```
npm install bcrypt
```

Теперь библиотека доступна нам для импорта:

```
const bcrypt = require("bcrypt");
```

Попробуем bcrypt на практике — напишем две функции, одна из которых будет хешировать пароль, вторая — сравнивать введённый пароль с хешем.

```
const bcrypt = require("bcrypt");

async function hashPassword(password) {
    const rounds = 10; // типичное значение: 10-12
    return bcrypt.hash(password, rounds);
}

// Использование
(async () => {
    const hash = await hashPassword("qwerty123");
    console.log(hash);
})();
```

На выходе была получена такая строка:

```
$2b$10$lwxEUgggbYEsB2fdcj8RcO/WImDDffX6t4Tf8.1EnjrAqFTMRkdNO
```

Теперь функция для проверки пароля:

```
const bcrypt = require("bcrypt");

async function verifyPassword(password, passwordHash) {
    return bcrypt.compare(password, passwordHash);
}

// Проверка
(async () => {
    const hash = await bcrypt.hash("qwerty123", 10);

    console.log(await verifyPassword("qwerty123", hash));
})();
```

В результате выполнения была получена истина:

```
true
```

Теперь проверим с неверным паролем:

```
const bcrypt = require("bcrypt");
```

```
async function verifyPassword(password, passwordHash) {
    return bcrypt.compare(password, passwordHash);
}

(async () => {
    const hash = await bcrypt.hash("qwerty123", 10);

    console.log(await verifyPassword("wrong", hash));
})();
```

На выходе получили ложь:

```
false
```

Таким образом, мы создали две функции, которые уже можно использовать для создания и проверки паролей в наших приложениях.

Сервер nodejs с авторизацией

Теперь напишем API, которое позволит нам создавать аккаунт и входить в него. В рамках простейшего примера реализуем только 2 соответствующих маршрута, пока не углубляясь в заголовки запросов, токены и cookies.

```
const express = require('express');
const { nanoid } = require("nanoid");
const bcrypt = require('bcrypt');

const swaggerJsdoc = require('swagger-jsdoc');
const swaggerUi = require('swagger-ui-express');

const app = express();
const port = 3000;

const swaggerOptions = {
    definition: {
        openapi: '3.0.0',
        info: {
            title: 'API AUTH',
            version: '1.0.0',
            description: 'Простое API для изучения авторизации',
        },
        servers: [
            {
                url: `http://localhost:${port}`,
                description: 'Локальный сервер',
            },
        ],
    },
```

```
    apis: ['./a.js'],
};

let users = [];

function findUserOr404(username, res) {
    const user = users.find(u => u.username == username);
    if (!user) {
        res.status(404).json({ error: "user not found" });
        return null;
    }
    return user;
}

async function hashPassword(password) {
    const rounds = 10;
    return bcrypt.hash(password, rounds);
}

async function verifyPassword(password, passwordHash) {
    return bcrypt.compare(password, passwordHash);
}

const swaggerSpec = swaggerJsdoc(swaggerOptions);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));

app.use(express.json());

app.use((req, res, next) => {
    res.on('finish', () => {
        console.log(`[${new Date().toISOString()}] [${req.method}]
${res.statusCode} ${req.path}`);
        if (req.method === 'POST' || req.method === 'PUT' || req.method ===
'PATCH') {
            console.log('Body:', req.body);
        }
    });
    next();
});

/**
 * @swagger
 * /api/auth/register:
 *   post:
 *     summary: Регистрация пользователя
 *     description: Создает нового пользователя с хешированным паролем
 *     tags: [Auth]
 *     requestBody:
 *       required: true
 *       content:
```

```
 *         application/json:
 *           schema:
 *             type: object
 *             required:
 *               - username
 *               - password
 *               - age
 *             properties:
 *               username:
 *                 type: string
 *                 example: ivan
 *               password:
 *                 type: string
 *                 example: qwerty123
 *               age:
 *                 type: integer
 *                 example: 20
 *     responses:
 *       201:
 *         description: Пользователь успешно создан
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 id:
 *                   type: string
 *                   example: ab12cd
 *                 username:
 *                   type: string
 *                   example: ivan
 *                 age:
 *                   type: integer
 *                   example: 20
 *                 hashedPassword:
 *                   type: string
 *                   example:
$2b$10$kO6Hq7ZKfV4cPzGm8u7mEuR7r4Xx2p9mP0q3t1yZbCq9Lh5a8b1QW
 *       400:
 *         description: Некорректные данные
 */

app.post("/api/auth/register", async (req, res) => {
    const { username, age, password } = req.body;

    if (!username || !password || age === undefined) {
        return res.status(400).json({ error: "username, password and age are
required" });
    }
```

```
    const newUser = {
        username: username,
        age: Number(age),
        hashedPassword: await hashPassword(password)
    };

    users.push(newUser);
    res.status(201).json(newUser);
});

/**
 * @swagger
 * /api/auth/login:
 *   post:
 *     summary: Авторизация пользователя
 *     description: Проверяет логин и пароль пользователя
 *     tags: [Auth]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required:
 *               - username
 *               - password
 *             properties:
 *               username:
 *                 type: string
 *                 example: ivan
 *               password:
 *                 type: string
 *                 example: qwerty123
 *     responses:
 *       200:
 *         description: Успешная авторизация
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 login:
 *                   type: boolean
 *                   example: true
 *       400:
 *         description: Отсутствуют обязательные поля
 *       401:
 *         description: Неверные учетные данные
 *       404:
 *         description: Пользователь не найден
```

```
 */

app.post("/api/auth/login", async (req, res) => {
    const { username, password } = req.body;

    if (!username || !password) {
        return res.status(400).json({ error: "username and password are
required" });
    }

    const user = findUserOr404(username, res);
    if (!user) return;

    isAuthentethicated = await verifyPassword(password,
user.hashedPassword);
    if (isAuthentethicated)
    {
        res.status(200).json({ login: true });
    }
    else
    {
        res.status(401).json({ error: "not authentethicated" })
    }
});

app.listen(port, () => {
    console.log(`Сервер запущен на http://localhost:${port}`);
    console.log(`Swagger UI доступен по адресу http://localhost:${port}/api-
docs`);
});
```

Теперь регистрация выглядит следующим образом:

Request body required

application/json

Edit Value | Schema

```
{
  "username": "Daria",
  "password": "qwerty123",
  "age": 2
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3000/api/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "Daria",
  "password": "qwerty123",
  "age": 25
}'
```

Request URL

```
http://localhost:3000/api/auth/register
```

Server response

Code	Details
201	Response body

```
{
  "username": "Daria",
  "age": 25,
  "hashedPassword": "$2b$10$HZONRa8co95IVDqJ.MhNQ.O4A5Jivb6hgy1.L0B.QgKkO21JXgZIW"
}
```

Download

Вход с верным паролем:

Request body required

application/json

Edit Value | Schema

```
{
  "username": "Daria",
  "password": "qwerty123"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3000/api/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "Daria",
  "password": "qwerty123"
}'
```

Request URL

```
http://localhost:3000/api/auth/login
```

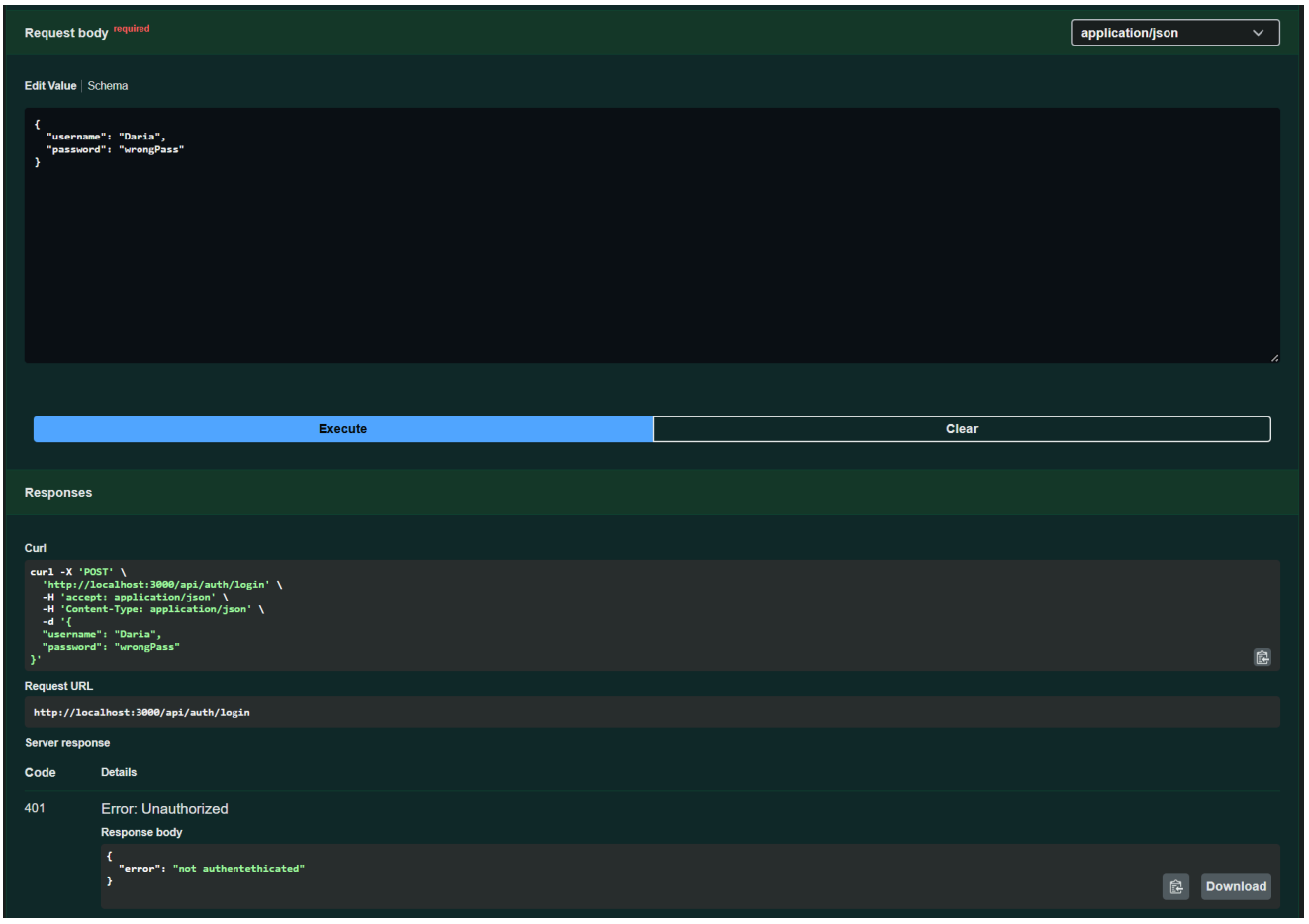
Server response

Code	Details
200	Response body

```
{
  "login": true
}
```

Download

Вход с неверным паролем:



Практическое задание

Необходимо создать серверное приложение на node.js, которое будет включать в себя следующие маршруты:

Маршрут	Метод	Описание
/api/auth/register	POST	Регистрация (создание) пользователя
/api/auth/login	POST	Вход в систему
/api/products	POST	Создать товар
/api/products	GET	Получить список товаров
/api/products/:id	GET	Получить товар по id
/api/products/:id	PUT	Обновить параметры товара
/api/products/:id	DELETE	Удалить товар

Минимальный перечень полей для сущности "Пользователь":

- id
- email
- first_name
- last_name
- password (хешировать)

(в качестве логина использовать `email`)

Минимальный перечень полей для сущности "Товар":

- id
- title
- category
- description
- price

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой. Ссылка подгружается в соответствующий раздел СДО: Задания для самостоятельной работы -> Практические занятия 7-8.